Histogram Equalization in OpenCL and NumPy

Case Study in Parallelizing Image Processing

H.Kirollos

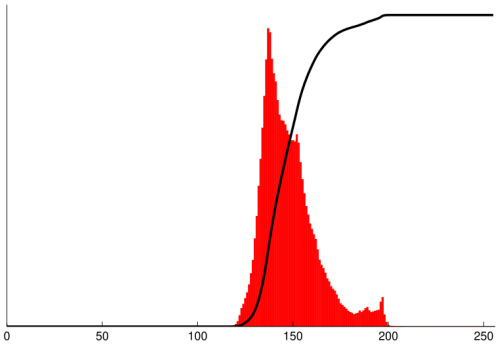
https://github.com/alonzo-bazaar/parallel-programming

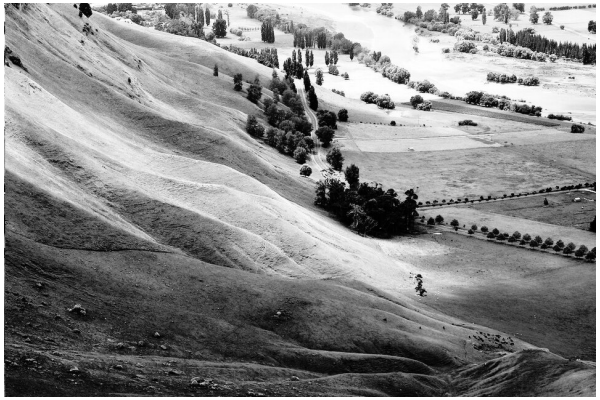
Contents

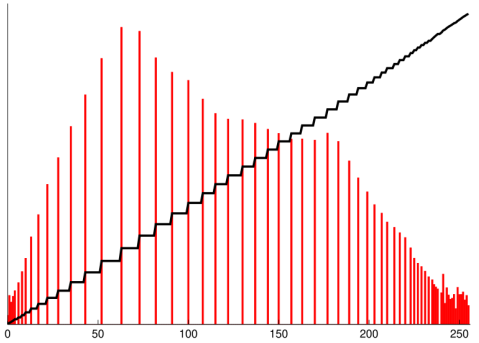
Histogram Equalization

Histogram Equalization

Image source: Wikipedia







Procedure

In general histogram equalization consists in:

- Compute histogram of image
  - (May be of whole image, of just one channel, of every channel...)
- Compute cdf (addition scan) of histogram
- Compute this value for every pixel

$$s_k = \text{round}\left(\frac{CDF(r_k) \cdot (L-1)}{n}\right)$$

(formula from an article on medium)

- optional: as a preprocessing step, an rgb image may be converted to a color space like HSL for processing, then converted back to rgb after processing

Our Procedure, and Parallel Patterns Therein

what we're doing is

- Convert the image from RGB(A) to HSL(A) (parallel pattern: **map**)
- Compute histogram of L channel of HSL(A) image (parallel pattern: **histogram**)
- Compute CDF of L channel (parallel pattern: **scan**)
- Normalize L channel of image using histogram CDF (parallel pattern: **map**)
- Turn HSL(A) image back into RGB(A) (parallel pattern: **map**)

Implementation

Technical Choices

- GPU version developed using Python and PyOpenCL
  - OpenCL for running locally (issues with remote development environments)

  closest alternative to CUDA, mimics most of its API
  - Python due to development timing constraints (C port was planned but could not be made in time)
  - Slides will still use CUDA terms (thread, thread block, grid, ...) instead of the corresponding OpenCL terms (work item, work group, ndrange, ...)
- CPU version developed using NumPy
  - Fairer compairison with GPU Python code than doing it in C or C++

  (any potential overhead from python runtime will be present in both the parallel and the sequential version)

OpenCL Implementation

PyOpenCL code laid out according to following pipeline

- host image is read from file, host histogram is initialized to all zeros
- host image and histogram are copied over to device
- device image is converted from RGB(A) to HSL(A) inplace
- histogram of L channel in device image computed into device histogram
- compute addition scan of device histogram inplace
- device image L channel normalized inplace

OpenCL Implementation: kernels used

- **Color space conversion**:
  - every thread acts on one pixel of the image, changing it inplace
  - 1d grid, image is treated as a flat array of pixels as no 2d specific behaviour is required due to the pixelwise nature of the operation

OpenCL Implementation: histogram kernel

- every thread acts on one pixel of the image
- thread blocks hold private partial histograms in local memory to avoid contention in global memory
- the first 256 threads in a block flush the partial sums to the global sum, after a barrier ensuring all elements have been counted in block local partial sum
- contention with multiple threads needing to increment values in the same array (histogram) handled through atomic operations
- 1d array, image treated as flat array of pixels, same reasoning as previous slide

OpenCL Implementation: histogram kernel code


```
__kernel void full_hist(const __global uchar* input,
                        __global uint* global_hist,
                        __local uint* local_hist,
                        const uint input_size,
                        const uint hist_size) {
    const uint gi = get_global_id(0);
    const uint li = get_local_id(0);
    if(li < hist_size) local_hist[li] = 0;
    barrier(CLK_GLOBAL_MEM_FENCE | CLK_LOCAL_MEM_FENCE);
    if(gi<input_size)
        atomic_add(&local_hist[input[gi]], 1);
    barrier(CLK_GLOBAL_MEM_FENCE | CLK_LOCAL_MEM_FENCE);
    if(li < hist_size)
        atomic_add(&global_hist[li], local_hist[li]);
}
```

OpenCL Implementation: addition scan kernel

- 2 level heirerachical scan, handles up to $\mathrm{max\_block\_size}^2$ elements
- kogge-stone and brent-kung versions have been made
- brent-kung used for both levels, though given input size (256 elements) the choice was arbitrary and had no performance impact
- OpenCL has no mechanism to have grid level syncrhonization, heirarchical scan threfore implmeneted as 3 kernels
- (code in next slide for one level of ks since bk code didn't fit)

OpenCL Implementation: addition scan code


```
__kernel void ks_block_scan(__global uint* global_data,
                            const uint global_data_size,
                            __local   uint* local_data,
                            const uint local_data_size) {
    const uint global_idx = get_global_id(0);
    const uint local_idx  = get_local_id(0);
    const bool is_active  = (global_idx < global_data_size);
    local_data[local_idx]=is_active?global_data[global_idx]:0;
    barrier(CLK_LOCAL_MEM_FENCE);
    for(uint stride = 1; stride < local_data_size; stride*=2) {
        const uint tmp=(local_idx>=stride)?local_data[local_idx-stride]:0;
        barrier(CLK_LOCAL_MEM_FENCE);
        if (local_idx >= stride) local_data[local_idx] += tmp;
        barrier(CLK_LOCAL_MEM_FENCE);
    }
    if(is_active) global_data[global_idx] = local_data[local_idx];
}
```

NumPy Implementation

NumPy code laid out according to following pipeline

- image is split into R, G, B (and A) channels (transposition then splitting)
- R, G, and B channels are “undiscretized” from `np.uint8` into `np.float`
- continuous R, G, and B channels are used to compute continuous H, S, and L channels (not inplace)
- L channel undergoes histogram normalization (requires discretization as an intermediate step)
- H, S, and normalized L channel are turned back into R, G, and B channels
- the normalized R, G, and B channels are used (together with the A channel) to construct the normalized image (requires another transposition)

This is a different set of operations from the PyOpenCL code but porting the degree of index fiddling done in OpenCL to NumPy proved problematic, requiring the two transpositions.

NumPy Implementation: code

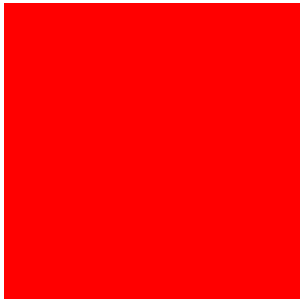
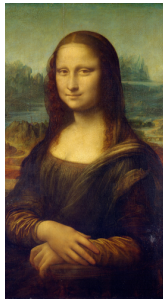

```
def continuous(ch:np.ndarray) -> np.ndarray:
    return ch.astype(np.float32)/256
def discrete(ch:np.ndarray) -> np.ndarray:
    return bound(np.floor(ch*256), by=255).astype(np.uint8)

# ...

r, g, b = np.transpose(image, (2, 0, 1))
r, g, b = map(continuous, [r, g, b])
h, s, l = rgb2hsl(r, g, b)
dl = discrete(l)
hist_l = np.histogram(dl.flatten(), np.arange(257))[0]
cdf_hist_l = np.cumsum(hist_l)
dl = bound(cdf_hist_l[dl] * 256 / dl.size).astype(np.uint8)
r, g, b = map(discrete, hsl2rgb(h, s, l))
out = np.stack([r, g, b])
```

Benchmarks

The PyOpenCL and the numpy versions have been tested on a small set of heterogeneous images to see how much would they take to equalize the image 10 times
The test images used are:

Benchmarks Code

Benchmarks Results

Time taken to equalize 10 image times

- Image Size: 421x748
- NumPy: 0.659s
- PyOpenCL: 0.023s
- Speedup: **28.64**

- Image Size: 100x100
- NumPy: 0.021s
- PyOpenCL: 0.012s
- Speedup: **1.642**

- Image Size: 11146x7479
- NumPy: 223.8s
- PyOpenCL: 4.863s
- Speedup: **46.03**

- Image Size: 796x1200
- NumPy: 2.016s
- PyOpenCL: 0.041s
- Speedup: **48.13**

- Image Size: 800x528
- NumPy: 0.835s
- PyOpenCL: 0.025s
- Speedup: **33.35**

Conclusions

In Conclusion

The PyOpenCL version is significantly faster in most cases. With a speedup 45 for larger images, and around 30 for smaller images.

The main outliar is the **1.642** speedup for the small flat red image, this is likely due to the image being very small, leading to high relative cost of sending the data over to gpu, being flat, there will also be high contention in the histogram kernel with all threads writing to the same address, leading to further loss of performance in the parallel version.